# Parallelized Simulation of Ship Motion and Interaction With Ocean Waves

Daniel Milan
dem3573@my.utexas.edu
EID: dem3573
TACC username: dmilan

Owen Scott
ocs326@my.utexas.edu
EID: ocs326
TACC username: ocs326

April 2026

## 1 Introduction

Modern naval architecture, like many fields in physical engineering, increasingly relies on computer modeling of ship designs in order to ensure safe and efficient designs and to enable cheap, fast iterative design processes that cannot be easily replicated with physical hardware. Our goal in this project is to create a software simulation pipeline as could be used in industry to that end; namely, we aim to develop a program for simulating a ship's hull under different ocean wave and weather conditions, thus granting a way to calculate the ship's motion over time in water. This requires a pipeline of several parts: generation of realistic wave data, generation of ship meshes and calculation of the forces affecting them and the waves, and computation of their interaction over time. Implementing this software allows for end-to-end analysis of newly created ship designs, so that users of this software would be able to design a ship and quickly gather data about its behavior in different ocean conditions to assess how well it performs.

Our wave data comes from the Joint North Sea Wave Project (JONSWAP) [3], a dataset of ocean wave spectra that detailes the distribution across different frequencies of ocean wave energy and is standard in literature for wave simulation. We use these spectra to generate realistic wave formulae and simulate the ocean interface over time, which provides a good known baseline within which we can add our simulated ship hull.

To create realistic ship meshes, we leverage data from the Series 60 Methodical Experiments [5], a set of ship-hull model tests developed at the David Taylor Model Basin and providing standardized information about different families of hull shapes. The hull shapes documented are industry-standard and widely used for CFD research, which makes them a good baseline for us to run experiments on. Specifically, the data we use to parameterize our meshes are the desired ship length (distance from bow to stern, i.e. front to back of the ship, on the $x$-axis), beam (width at the widest point, on the $y$-axis), draft (vertical distance

from keel to waterline, i.e. maximum depth beneath the water surface, on the $z$-axis), and the block coefficient $C_b$. $C_b \in (0, 1)$ represents how much the ship resembles a block, with $C_b < 0.7$ for sleeker hulls and $C_b > 0.7$ for fuller, slower hulls, though we constrain our experiments to run on the Series 60 data's subset of $C_b \in \{0.6, 0.65, 0.7, 0.75, 0.8\}$, a balanced spread of coefficients. We reconstruct the Series 60 geometry definitions into surfaces and then convert them into computational meshes which we can run further analysis on.

Upon generation of the mesh we will compute hydrostatic coefficients, added masses, and radiation dampings. These are physical properties which allow us to ultimately calculate the Response Amplitude Operator (RAO) for the hull mesh given our wave conditions. The RAO is a (complex-valued) frequency-domain transfer function $\text{RAO}(\omega, \beta) = \frac{X(\omega,\beta)}{A}$ where $X$ is the response amplitude, $A$ is the wave amplitude, $\omega$ is the wave frequency, and $\beta$ is the heading angle.

This allows us to compute the 6 degrees of freedom (surge, sway, heave, roll, pitch, yaw) based on the hull geometry and hydrostatic coefficients offline, by having the simulator for the ship motion simply query the precomputed RAO table and run it against all incoming wave components to quickly simulate ship motion. The end result of all of this is that we can simulate ship motion within ocean wave conditions over some time range, which accomplishes the goal for the scope of this project.

Overall, we expect to see reasonable output at every stage of the pipeline, since we will be using data collected from real physical hull measurements and wave spectra that we know to be a good baseline. We should also expect our final visualization of our ship movement to pass a basic "sanity check", i.e. the way the ship moves in the water should conform with how we expect it to do so.

# 2 Implementation

Broadly, the operation of our software pipeline establishes simulated data for both our waves and our ship mesh's reaction to the waves, then parallelizes the computation to model the ship's position over time. We first run a wave generation program to sample a number of waves in a given weather condition from the JONSWAP wave spectrum and produce properties for each of the waves (including amplitude, wavelength, angular frequency, phase shift, and direction in $x$ and $y$). We also have a program that constructs a mesh according to Series 60 data and tests it using the generated wave properties to compute hydrostatic coefficients and the 6 degrees of freedom of the waves for each wave direction and frequency. Finally, we take in generated data from the previous steps and use it to compute the ship's position over time.

## 2.1 Wave Simulation

Ocean surface geometry is generated using the JONSWAP (Joint North Sea Wave Project) spectral model, which describes the distribution of wave energy

across frequencies for a given sea state as per the equation:

$$S(f) = \alpha g^2 (2\pi)^{-4} f^{-5} \exp\left(-\frac{5}{4}\left(\frac{f_p}{f}\right)^4\right) \gamma^{\exp\left(-\frac{(f-f_p)^2}{2\sigma^2 f_p^2}\right)}.$$

The spectrum is parameterized by a peak frequency, an energy scale factor $\alpha$, and a peak enhancement factor $\gamma$, with three presets (CALM, MODERATE, and STORMY) covering different ocean conditions. From the spectral density, 100 wave components are sampled on a log-spaced frequency grid, each assigned an amplitude via $A_n = \sqrt{2S(f_n)\Delta f}$, a wavelength from the deep-water dispersion relation $\lambda = \frac{g}{2\pi f^2}$, a random phase, and a propagation direction drawn from a cosine-power spreading function centered on a dominant heading. The ocean surface height at any point $(x, y, t)$ is then the superposition of all 100 sinusoidal components, evaluated as

$$\eta(x, y, t) = \sum_{n=1}^{N} A_n \sin\left(\frac{2\pi(\hat{d}_n \cdot [x, y])}{\lambda_n} - \omega_n t + \phi_n\right).$$

This is easily parallelizable across space and time, making MPI well-suited to this task. Figure 1 illustrates a frame of the simulated wave heights for a section of ocean under stormy weather conditions. This particular simulation uses a grid resolution of $1000 \times 1000$ points spaced 5 cm apart, for a total simulated area of $50 \times 50$ m.

See the jonswap formulation from our code:

```
inline std::vector<double> jonswap_spectrum(
      const std::vector<double> &freqs,
      double fp, double alpha, double gamma_val)
  {
    std::vector<double> S(freqs.size());
    for (size_t i = 0; i < freqs.size(); ++i)
    {
      double f = freqs[i];
      double sig = (f <= fp) ? 0.07 : 0.09;
      double r = std::exp(-((f - fp) * (f - fp)) / (2.0 * sig * sig * fp * fp));
      S[i] = static_cast<double>(
          alpha *
          G *
          G *
          std::pow(f, -5.0) *
          std::pow(TWO_PI, -4.0) *
          std::exp(-1.25 * std::pow(fp / f, 4.0)) *
          std::pow(gamma_val, r));
    }
    return S;
  }
```

And the generation for all waves (ignoring the jonswap sampling setup code)

```
inline std::vector<Wave> generate_waves(
    JonswapConditions conditions,
    double dominant_dir_x = -1.0,
    double dominant_dir_y = -1.0,
    int n_waves = NUM_WAVES,
    uint32_t seed = std::random_device{}())
{
  std::vector<Wave> waves(n_waves);
  for (int i = 0; i < n_waves; ++i)
  {
    double f = freqs[i];
    double amplitude = std::sqrt(2.0 * S[i] * df[i]);
    double wavelength = G / (TWO_PI * f * f);
    double k = TWO_PI / wavelength;
    double ang_freq = std::sqrt(G * k);
    double phase = phase_dist(rng);
    double dev_deg = cosine_deviation_deg(spreads[i], rng);
    double dir_deg = dev_deg + dominant_angle_deg;
    double dir_rad = dir_deg * static_cast<double>(M_PI) / 180.0;

    waves[i] = Wave{amplitude, wavelength, ang_freq, phase,
        std::cos(dir_rad),
        std::sin(dir_rad),
    };
  }
  return waves;
}
```

And the function to get our wave height for any point x, y at time t:

```
inline double height_at(const std::vector<Wave> &waves, double x, double y, double t)
{
  double total = 0.0;
  for (const auto &w : waves)
  {
    double dot = (w.dir_x * x + w.dir_y * y) / w.wavelength;
    double angle = TWO_PI * dot - w.angular_freq * t + w.phase;
    total += w.amplitude * std::sin(angle);
  }
  return total;
}
```

Once the properties of the waves have been established, we use MPI to parallelize computation of the wave heights among multiple cores. Since the section of the ocean under consideration is modeled as a 2D matrix of points

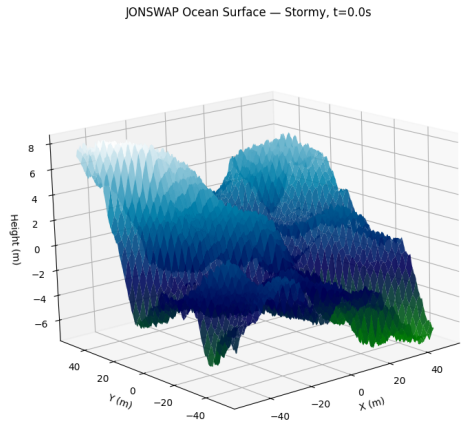


Figure 1: Visualization of a frame from the JONSWAP ocean surface simulation with stormy weather conditions and a grid resolution of $1000 \times 1000$ points spaced 5 cm apart.

in space, the problem lends itself easily to parallelism, and we take advantage of this by using MPI to distribute rows of the matrix evenly to different cores. Each core receives a set of assigned rows and all relevant wave data for those rows. We maintain a barrier so all cores begin their task at the same time, and once the barrier is down all cores compute the wave heights for the rows they have been assigned, forming the overall height matrix when the second barrier is passed. Computation time between barriers is assessed in Section 3.1 and illustrated in Figure 4.

We choose to use MPI collectives for our parallel implementation of the wave simulator because it best models the nature of the problem in that each core has its own task to perform, with its own associated data, and the tasks are independent of adjacent ones but together form the total height matrix for all waves. Point-to-point is unnecessary because all knowledge about height can be calculated from the provided wave data and does not require communicating with cores working in adjacent rows.

## 2.2 Hull Meshing and Hydrostatic Coefficient Computation

Our ship hull mesh is manually constructed in Python using digitized Series 60 offsets and parameterized with appropriate constants. An example ship hull mesh using a moderate set of parameters is pictured in Figure 2. Note that the visualization software used to generate the image uses triangular faces but the original constructed mesh is made with quadrilateral faces.

For computation of hydrostatic coefficients, we use the Capytaine Python

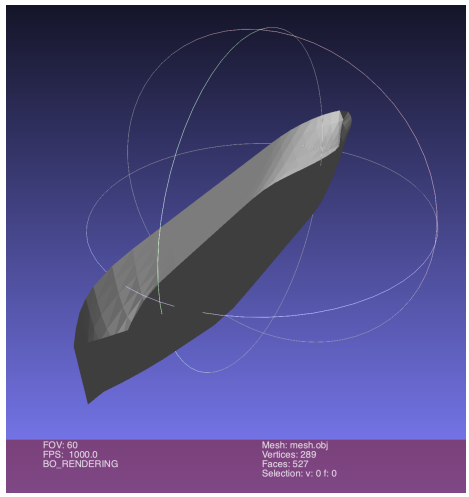


Figure 2: A ship hull mesh constructed from Series 60 data with a length of 100 m, beam of 15 m, draft of 10 m, and block coefficient $C_b = 0.80$.

library [1][2], an up-to-date Python implementation of the FORTRAN boundary element solver software NEMOH [4]. (We avoid reimplementing the functionality offered by Capytaine from scratch because calculation of hydrostatic coefficients is fast on a single thread and so not conducive to parallelization, Capytaine and NEMOH are standard libraries in the literature and it would be unproductive to reimplement them, and solving for mesh properties is out of scope for this project.) Both of these software tools are used for modeling how floating bodies such as ships interact with wave physics, and use the Boundary Element Method (BEM) which only models surface elements and thus is more efficient than full CFD solvers. We use Capytaine to take the digitized Series 60 offsets and create ship hull meshes, as well as compute hydrostatic coefficients which we can use to calculate the forces acting on the ship. The output of our Capytaine software contains data for all hydrostatic coefficients, rigid body degrees of freedom, and for each degree of freedom, the added masses and radiation dampings for all tested wave frequencies based on the data found in the previous step using the JONSWAP wave spectrum. No parallelism is used in this step of the pipeline, which plays more of an auxiliary role in the context of the project.

## 2.3 Ship Movement Simulation

Given the precomputed wave components and hydrodynamic RAO data, `ship_motion.cpp` computes the six degree-of-freedom rigid body motion of the ship over a 60-second simulation window at 20 Hz resolution, writing the results to `boat_motion.txt`. For each DOF $k \in \{\text{surge, sway, heave, roll, pitch, yaw}\}$, the displacement at time $t$ is evaluated as a superposition over all $N$ wave components:

$$x_k(t) = \sum_{n=1}^{N} A_n \cdot |H_k(\omega_n, \theta_n)| \cdot \cos(\omega_n t + \phi_n + \angle H_k(\omega_n, \theta_n))$$

where $H_k(\omega_n, \theta_n)$ is the complex RAO interpolated to wave $n$'s angular frequency and heading, loaded from the Capytaine BEM output. The RAO magnitude and phase are precomputed into a lookup table indexed by wave.

This summation is parallelized with OpenMP across the $N = 100$ wave components, with a `reduction` clause accumulating contributions into the six-element motion array:

```
#pragma omp parallel for reduction(+:motion[:6]) schedule(guided)
for (int n = 0; n < (int)waves.size(); ++n) {
    double phase_total = waves[n].angular_freq * t + waves[n].phase;
    for (int k = 0; k < 6; ++k)
        motion[k] += waves[n].amplitude * rao.mag[k][n]
                   * std::cos(phase_total + rao.phase[k][n]);
}
```

We parallelize over waves rather than DOFs because the wave axis ($N = 100$) provides sufficient work to saturate available threads, whereas the DOF axis (6 elements) is too coarse for effective distribution. We evaluate both `static` and `guided` scheduling. In practice `static` outperforms it here because the problem is too small. The overhead of `guided` exceeds any load-balancing benefit it provides, which is clearly displayed in the results.

# 3 Results

Overall, our simulator is fairly successful at simulating natural movement of ship hulls within the input bounds we have given it. An example of our output visualization is in Figure 3, where the ship hull interacts with the ocean waves as prescribed by the data generated throughout the program.

The output that we get from every part of the software seems to be in line with both our expectations as to what it would produce when we developed the software, as well as basic sanity checks for physical behavior of watercraft. Our software performance is also relatively quick, in part due to the parallel sections of our code which allow computation to proceed faster for larger problem instances.

## 3.1 Wave Simulation

Our JONSWAP wave simulation framework offers a speedup as expected when run in parallel with more cores, as depicted in Figure 4. The top-left graph illustrates how time elapsed changes as more cores are used to run the computation, the top-right graph illustrates how the speedup compares to an ideal linear speedup, the bottom-left graph shows the parallel efficiency versus the number

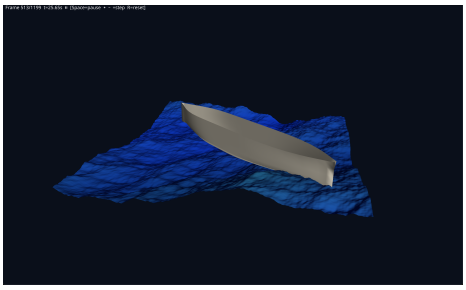

Figure 3: A frame from the visual animation of the ship movement in the ocean. The ship hull has $C_b = 0.70$ and the ocean is under stormy conditions.

of cores, and the bottom-right graph shows the parallel efficiency versus the remaining data when the 1000 rows are divided among cores (for example, with 35 rows, the remaining data is 1000 mod 35 = 20 rows).

The time elapsed with MPI drops below 500 ms at 10 cores used and stays very low compared to the serial time average of over 3.5 s, but as is evident in the graphs comparing to number of cores, the speedup and efficiency fluctuate as the number of cores varies The reason for this is clear from the very strong relationship in the bottom-right graph, which is that a higher remaining amount of data produces a lower efficiency overall. This is likely due to the fact that when the data is split, evaluation done by any one node on its own data is relatively fast and not exceedingly large compared to the work set to be done on any data that did not split evenly among nodes, so the extra work done on the remainder of the data is noticeable when compared to the time for the fully parallel work, thus producing a notable impact on efficiency and speedup. Splitting a much larger amount of data would likely avoid this problem, since the remaining data would take much less time to process compared to the larger quantity of data processed in parallel, thus keeping the overall speedup and efficiency closer to ideal.

## 3.2 Ship Movement

The parallel performance of our ship motion simulation code is depicted in Figure 5 (for static scheduling of threads) and Figure 6 (for guided scheduling of threads). In both of these scheduling strategies, we can see that multithreading does offer a speedup, although its performance boost is much less than ideal and still significantly less than that achieved by the parallelism implemented for the JONSWAP wave simulation code.

This is likely due to the fact that the amount of data being operated on is not enough to productively share between threads, meaning the overhead of operating multithreading induced by OpenMP ends up being a nontrivial fraction of the time it takes to run the program, and thus the speedup and efficiency are far from theoretical. Speedup is actually worse overall for guided scheduling

Figure 4: Performance graphs for running the JONSWAP wave simulation software. The graphs generally show the expected speedup over serial computation with ideal data division, though a larger remainder from data division produces markedly worse results.

as opposed to static scheduling, again likely because it adds increased overhead for thread management while yielding little gain in parallel computation on the comparatively small amount of data available for operating on. That being said, both scheduling strategies on all numbers of multiple threads tested achieves a noticeably lower time to run computations versus that of the single-threaded version of the program, showing that the lack of near-ideal speedup is most plausibly the quantity of data and not any issue with the way the problem itself is being modeled and solved.

# 4 Conclusion

Overall, our simulation framework is largely successful in modeling ship movement through an ocean environment. Our use of industry-standard data and modeling of the most important factors in determining ship movement gives us

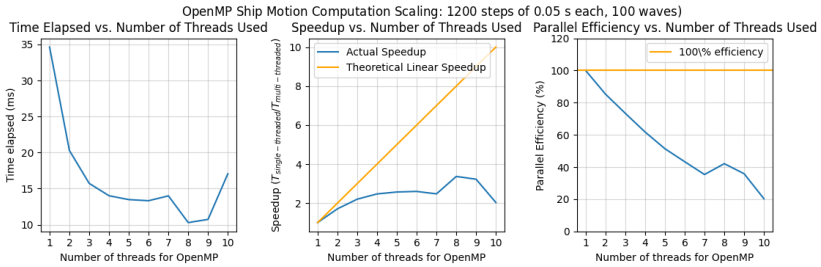


Figure 5: Performance graphs for running the ship motion simulation software with static thread scheduling.

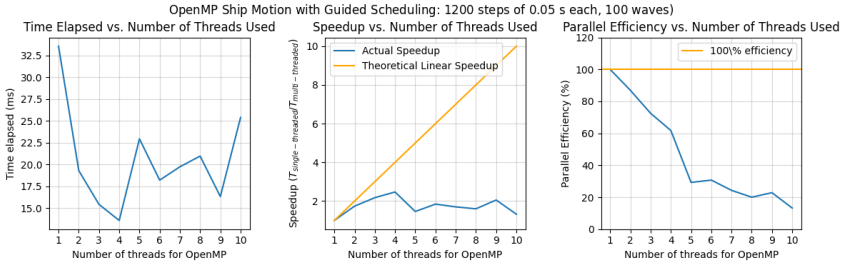


Figure 6: Performance graphs for running the ship motion simulation software with guided thread scheduling.

a reliable picture of how standard watercraft would behave in ocean conditions. Simulations like these can productively be run when developing vessels to assess their stability and safety, so our models, while not as accurate as full real-world testing, serve as a way to gain quick data on computer hull models and conduct iterative design more knowledgeably.

While our simulation framework is a fairly faithful model of the ocean environment as according to the data we have available, we still make approximations which could affect the overall accuracy of the data and results. Most pertinently is that the ships we simulate are static, rigid bodies, meaning that our simulator cannot account for ships with moving parts such as propellers or rudders, nor can they account for deformations in materials, the former of which especially limits the scope of the watercraft to which our simulation can be meaningfully applied. Complicated control surfaces, while theoretically possible to mesh, may also react to ocean conditions in ways that would be more faithfully represented with a full CFD analysis of the system, rather than just the BEM solver with which we compute hydrostatic coefficients for our ship meshes. Finally, other environmental effects like wind could significantly affect the behavior of waves and the movement of the ship, especially for lighter watercraft in stormy conditions where airflow can exert nontrivial forces.

This project can naturally be extended by accounting for the above. Incorporating a Finite-Element Method (FEM) solver would allow for solving for deformation of the ship hull and would produce a more accurate simulation. This extension would naturally be parallelizable, as the majority of computation would be the solution of massive matrix systems, a well-documented embarrasingly parallel problem in literature. More complicated ship meshes could also be represented depending on accessible data, and modifications to the simulator could feasibly enable moving parts on the ship hull to be modeled.

# References


[1] Matthieu Ancellin and Frédéric Dias. "Capytaine: a Python-based linear potential flow solver". In: *Journal of Open Source Software* 4.36 (Apr. 2019), p. 1341. DOI: `10.21105/joss.01341`. URL: `https://doi.org/10.21105%2Fjoss.01341`.

[2] Aurélien Babarit and Gérard Delhommeau. "Theoretical and numerical aspects of the open source BEM solver NEMOH". In: *Proceedings of the 11th European Wave and Tidal Energy Conference (EWTEC2015)*. Nantes, France, 2015.

[3] K Hasselmann et al. *Measurements of wind-wave growth and swell decay during the Joint North Sea Wave Project (JONSWAP)*. Tech. rep. 8. 1973, pp. 1–104.

[4] Ruddy Kurnia et al. "NEMOH: Open-source boundary element solver for computation of first-and second-order hydrodynamic loads in the frequency domain". In: *Computer Physics Communications* 292 (2023), p. 108885.

[5] Frederick Henry Todd. *Series 60 Methodical Experiments with Models of Single-Screw Merchant Ships*. Report / David Taylor Model Basin 1712. Washington, D.C.: David Taylor Model Basin, 1963.